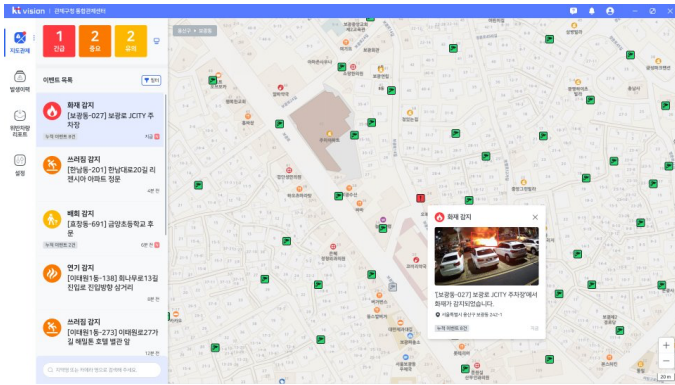


CCTV 자동 관제 솔루션



개요: 고객의 CCTV 영상을 받아 이벤트(화재, 연기, 위험지역 접근 감지 등)를 감지 후 알림 발생시키는 시스템

프로젝트 기간: 2023.11 ~ 2025.07 (퇴사 후에도 2주간 진행) / 팀구성: 4명(팀장 1명, 기획+디자인 1명, 개발+현장지원 2명)

사용기술: DeepStream, Docker, Bash, Jenkins, PyArmor, PyTorch, OpenCV, Flask, MariaDB, Multi-processing, Python

배경

- CCTV 관제 시스템을 인수 후 유지·납품하는 사업 진행 (인수 당시 오탐 빈도 높음, 다수의 버그/오류 존재)
- 주요 고객은 공공기관으로 설치·점검 환경 제약(폐쇄망, 관제센터 출입 제한) 존재.
- 유사한 업체가 많아서 기술 유출의 우려 존재.
- 제어용 Web 페이지는 외부 SI 업체에서 관리

역할

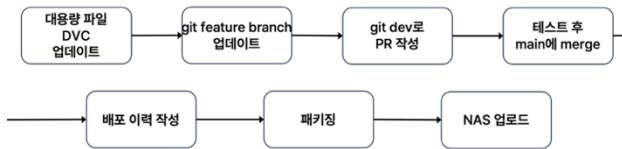
- 오탐 제거를 통한 감지 성능 향상
- 폐쇄망, 온프레미스 환경에 맞춘 배포·설치·백업 규칙 수립 및 자동화
- 인수 후 불안정한 시스템 안정화(버그 수정, RCA 기반 문제 해결)
- 난독화 및 라이선스 기반 인증 체계 수립 및 제품 스펙화

주요 진행 사항

- 이미지 유사도 비교를 통한 오탐 검출 알고리즘 개선을 통한 오탐 제거 성능 개선



- 객체 영역 Crop, Tracking 알고리즘 도입으로 F1-score 81.94→84.34, 클립당 분석 속도 4ms → 0.9로 개선

- **배포 파이프라인 설계 및 구현**

- 고용량 파일 관리(DVC), 난독화와 패키징(Pyarmor), 환경제어(Docker), 자동화가 적용된 배포 과정 표준화
- (퇴사 당시 진행 및 검토 중이던 사안) Jenkins를 활용해 DVC, git에 변화가 감지될 경우 테스트 후 패키징 하는 과정 자동화, Terraform을 활용한 IaC, 서버 가상화

- RCA로 19개의 이슈 해결 후 트러블 슈팅 문서 제작

```
if current_process().name == "MainProcess":
    self._logger.info(f'{current_process().name}: vision Tear Down called. by sig= {sig}, sig_name= {sig_name}')
    self.tear_down()
else:
    self._logger.info(f'{current_process().name}: vision Tear Down called. by sig= {sig}, sig_name= {sig_name},
```

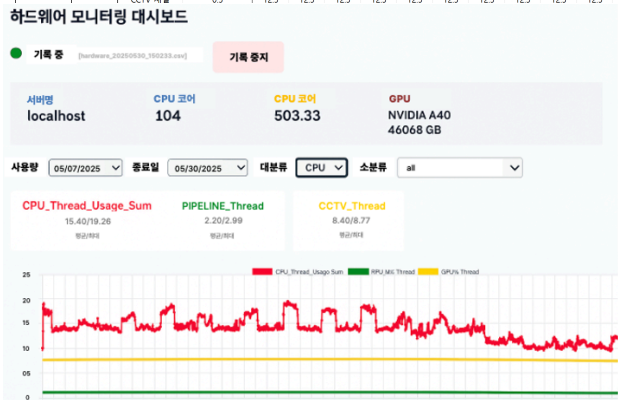
- 예: Multi-process 환경에서 자식 프로세스 종료 시 부모 프로세스까지 종료되는 문제 발생
→ Signal 발생 시 프로세스 명을 확인하는 분기 처리를 통해 해결
- 현장 관리 시 동일 이슈가 발생할 경우를 대비하기 위해 이슈 해결 내역을 트러블 슈팅 문서로 정리

- 백업 자동화 구축

Backup & Restore 항목 리스트업						
위치	항목	항목 상세	위치 상세	연관 DB 테이블	DB 설정	백업 여부 테스트
Admin Web	카메라 목록	카메라 & 영. 설치 지역(시/도) 카메라 유형, 위도, 경도, 상세주소, PTZ유형, 카메라 IP, RTSP URL, PTZ 카메라 ID&PW, 일괄삭제 주소 / 포트번호, 분석서버 이름, 설명	카메라 > 카메라 & 배치 > 카메라 관리	aw_cctv aw_cctv_stat_info	CCTV 목록 CCTV 관련 동작 이력	카메라 목록을 역설을 Export 백업 이후 리스토어 된 카메라 목록과 H
Admin Web	사용자 목록	사용자 아이디, 이름, 휴대폰번호, 권한 등	설정 > 계정 설정 > 사용자	core_user_ext core_user_pass	사용자 목록 및 연락처 사용자 비밀번호	계정 설정에 Test 계정을 만든 후 역설을 백업 이후 운영 SW에 Test 계정으로 로
Admin Web	서버 목록	서버명, 서버아이디, 해당 서버에 등록된 채널	서버 > 서버 관리 > AI 서버	aw_device aw_device_info	연결된 서버 서버의 동작 이력	서버 관리에 등록된 서버를 확인 백업 이후 등록된 서버가 있는지 확인
Admin Web	지도 축적 Default값	납품 지역별 상정한 지도 축적 사용. 지도 화면 진입 시 초기 축적값	카메라 > 카메라 & 배치 > 지도 배치 관리	없음		지도 축적 값을 변경 한 후 백업 리스토어 후 변경된 지도 축적 값이 보
Admin Web	지도 중앙값	납품 지역별 상정한 중앙값(중경도) 사용. 지도 화면 진입 시, 초기 중앙값 위경도	카메라 > 카메라 & 배치 > 지도 배치 관리	없음		지도 중앙 값을 변경 한 후 백업 리스토어 후 변경된 지도 중앙 값이 보
Admin Web	기권	일반설정에 설정된 기권 명	설정 > 일반 설정 > 기권	auth_menu - ADWE042300(백업 불필요) aw_area - area_type = ROOT	헤더를 비롯한 운영 SW에서 관리 하는 메뉴 관리 지역(시/도)	기권을 변경한 후 백업 리스토어 후 변경된 기권 값이 보이는

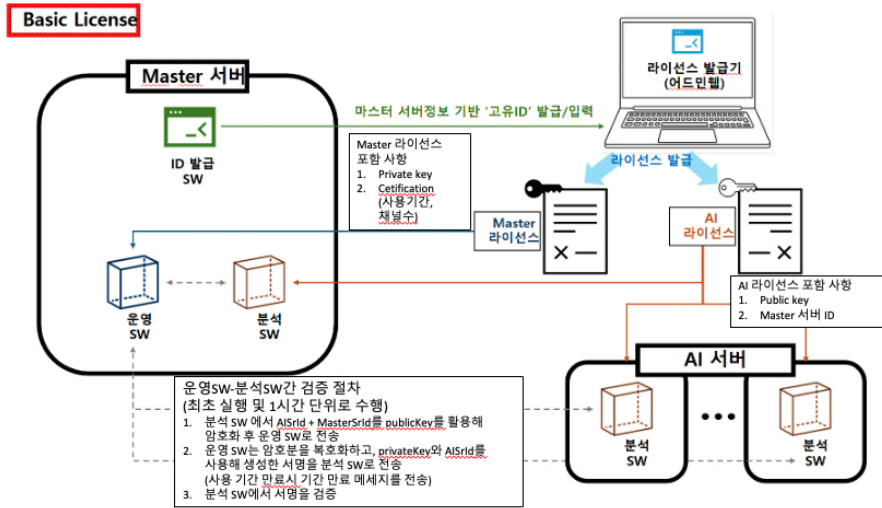
- 백업 항목 리스트업 후 Cron, Bash 스크립트를 활용하여 주기적으로 백업되는 프로세스 구성

- 채널당 사용하는 하드웨어 리소스 계산, 실행 테스트 및 모니터링 툴 제작

[illegible]

- 안정성 확보를 위해 시스템의 채널당 하드웨어 사용량을 계산 및 테스트 후 권장 스펙에 대한 문서를 작성
- 현장 대응을 위해 모니터링이 필요 했음. 약 1주일의 시간을 들여 하드웨어 모니터링 툴을 제작
- Deepstream 개선을 통해 허용 가능 채널수 +3개 확보
- (퇴사 당시 진행) Prometheus, Grafana 도입

- 채널 수, 사용기간을 기반으로 한 License 인증 로직 설계 및 구축



성과

- 오탐 처리 알고리즘 정탐 90장 테스트 데이터 기준 아래 성능 달성

항목	Before	After
F1-score	81.94	84.34
분석속도(per clip)	4ms	0.9ms

- 배포·설치·백업 과정 표준화 및 자동화를 통해 엔지니어 개입 최소화 및 운영 안정성 확보
- 점검 단위인 30일간 무중단 운영, 트러블 슈팅 매뉴얼
- 운영 및 구매 가이드로 활용될 채널당 리소스 요구 스펙 문서화 하드웨어 스펙 모니터링 툴
- A40 GPU 기준 허용 가능 채널 25 → 28
- 채널 수, 사용 기간 기반 과금 체계 지원

주차 관제 솔루션



개요: 주차장 입구의 아일랜드의 차량 입차 확인 후 출입구 개폐, 번호판 위변조 탐지 및 차번 인식을 하는 솔루션

프로젝트 기간: 2022.04 ~ 2023.02 / 팀구성: 7인(팀장 1명, CV개발자 3명, Web 개발자 2명, 비즈니스 1명)

사용기술: PyTorch, OpenCV, pandas, ONNX, TensorRT, Python, C++

데모 영상 링크: <https://youtu.be/dtFXy78bGZo>

배경

- NIPA 정부 과제로써 인공지능 번호 인식 기술을 가진 공급업체에서 목표치 이상의 성능, 조건을 맞추어 제품을 제작한 후 수요 기업으로 공급하는 과제
 - Real Time으로 들어오는 차량, 객체에 대한 분석이 가능해야 함.
 - 3D 카메라를 활용한 기술 연구를 위해 3D 카메라를 사용해야 함.
기존엔 위변조 인식에 3D 카메라를 사용하려 했으나 어려움을 증명한 후 객체 인식에 사용하기로 결정
 - 원가 저감 및 설치 편의성을 위해 Edge Device를 활용해야 함.

역할

- 번호 인식 시스템 구축
 - 번호 인식 정확도 95% 이상 달성
 - ToF 카메라를 활용한 객체 인식 정확도 95% 이상 달성
 - 하드웨어 연구 - ToF, RGB 카메라, Edge Device(Jetson Nano)

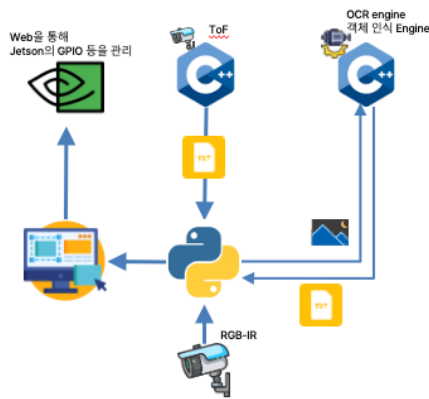
주요 진행 사항

- 번호 인식 엔진 개발



- 배경(간판, 현수막, 전단지 등)의 문자를 인식하는 문제를 자동차 객체 영역 탐지 및 이미지 하단 절반에서 번호판을 탐지하는 방식으로 해결
- 기존 상용화된 OCR로는 실제 데이터를 테스트 해본 결과 60%도 미치지 못하는 정확도를 보임.
객체 인식을 통해 번호를 찾은 후 순서대로 이어 붙이는 알고리즘 사용시 89%의 성능에 도달
3장의 프레임에 있는 번호들을 Voting 알고리즘을 통해 조합하여 최종 번호를 인식하는 방식으로 목표 정확도인 95%를 넘는 96.4%를 달성

- 시스템 구축



- ToF 카메라는 C++ SDK만 제공했으며, RGB 카메라는 Python SDK만을 제공했음.
이를 해결하기 위해 중앙 시스템은 Python을 통해 구축, ToF를 통한 인식 결과를 txt 파일에 기록,
txt 객체 유무 판독 결과에 변경 사항이 생길 시 중앙 Python 시스템에서 이를 인식하는 방식으로 해결
- Jetson Nano라는 제한된 환경 탓에 이미지 분석 속도가 충분하지 못함.
이를 TensorRT를 통해 최적화 하여 이미지 1장 당 평균 분석 시간을 0.22초 → 0.05초로 줄임
- 인식된 결과에 따라 주차장 입구 출입 통제 바를 개폐하는 기능이 필요했음.
GPIO 기능을 연구, Web 팀과 협업하여 제어용 웹페이지를 구현.

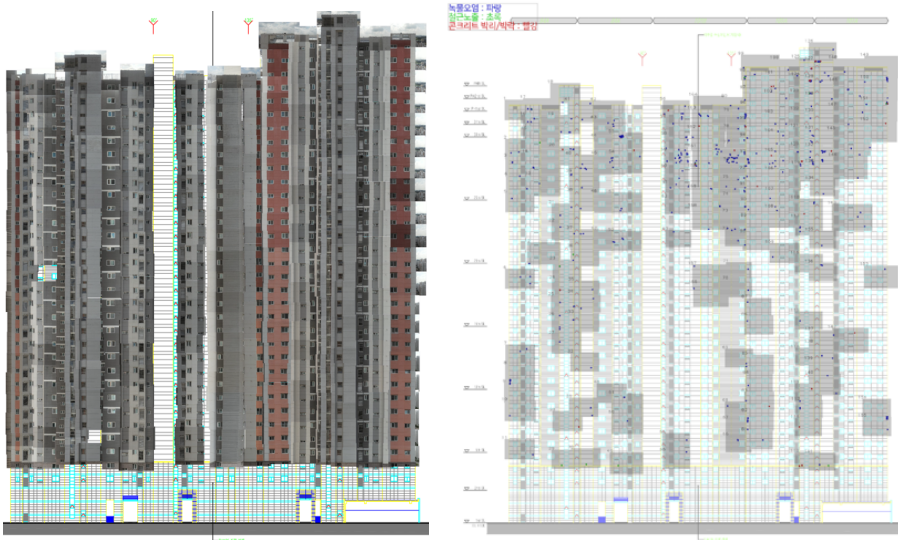
성과

- 객체 인식 95.3% (ToF를 활용한 물체 유무 판별 정확도 100%)
- 번호 인식 Accuracy: 60% 미만 → 96.4%
- Jetson Nano 환경에서 3장의 이미지에서 객체 인식부터 번호 인식까지 총 수행시간 0.151초

건축물 결함 검사 솔루션



사진번호.1	
균열번호	길이
1	0.02m
2	0.03m
3	0.03m
4	0.2m
5	0.04m
중계	0.33m



개요: 건축물 내 결함을 검사한 후 상세 검사 결과 내역, 전체 건축물내 결함 정보를 담은 레포트를 만들어 주는 솔루션

프로젝트 기간: 2022.03 ~ 2023.11

팀구성: 21명(CTO 1명, CV개발자 6명, Web 개발자 6명, 비즈니스 5명, 기획 2명, 디자인 1명)

사용기술: PyTorch-Segmentation&OD, TorchVision, OpenCV-Stitching&Localization, pandas, Flask, scikit-learn, Python

배경

- 기존 건축물 결함 검사는 사람이 수작업을 통해 진행되고 있었는데, 이를 드론, AI를 통해 해결하는 솔루션을 만들려 함.
건축물 결함 검사 시 필요한 고객사의 요구조건은 아래와 같음.
 - 전체 구조물상 균열의 위치가 어느 곳인지 필요
 - 건설공사 안전관리 지침 3항에 따르면 균열의 두께 기준은 0.3mm이므로 두께가 0.3mm 인 균열들을 찾을 수 있어야 한다.
 - 균열의 길이, 두께를 자동으로 측정할 수 있어야 한다.
- 최초 합류 당시 긍정적인 기술 검토를 통해 사업성이 보여 사업을 시작하던 단계.

역할

- 학습 파이프라인 설계, 성능 고도화
- Metadata를 포함한 8k 이미지, 이미지 내의 결함 위치, 도면, 촬영 순서가 주어졌을 때, 전체 건축물 내 결함의 위치를 추정하는 알고리즘 개발 (Stitching, Localization, VO)
- 균열 두께 측정 알고리즘 개발

주요 진행 사항

- 초기 파이프라인 설계

입사 당시 정형화된 Training 포맷이 부재했으며, Metric을 활용한 평가도 존재하지 않았음.

"Dataset → DataLoader → Train(Model Class) → Valid with Metric"으로 이어지는 초기 베이스라인을 구성 Opening, CRF 알고리즘을 활용해 정확도 개선

- Stitching을 활용한 전체 구조물 상 균열의 위치를 찾는 알고리즘이 가진 문제 해결

- 고용량 이미지를 사용해서 작업이 어려움. → tiff 파일, 이미지 분할로 해결
- 건물의 1층이나 2층이나 비슷하게 생겨서 이미지가 붙는 지점을 정확히 하기 어려움.
→ Stitching 시 건물면의 일부만을 Slicing 하여 사용
- Drone의 짐벌이 건물면과 완벽하게 수직이 되기 어려움. → 물리적으로 해결 불가
- 아무리 캘리브레이션을 진행해도 실제 이미지에 왜곡이 있을 수 밖에 없고
고층으로 갈수록 폭이 좁아지는 양상이 생김.



- Localization을 활용한 전체 구조물 상 균열의 위치를 찾는 알고리즘 개발 → 논문 참고

<https://github.com/ksm0517/portfolio/blob/main/GPS%EB%A5%BC%20%ED%99%9C%EC%9A%A9%ED%95%9C%20%EA%B1%B4%EC%B6%95%EB%AC%BC%20%EB%82%B4%20%EA%B7%A0%EC%97%B4%20%EC%9C%84%EC%B9%98%20%ED%91%9C%EA%B8%B0%20%EC%95%8C%EA%B3%A0%EB%A6%AC%EC%A6%98.pdf>

- 실내 환경에서 동작 시킬 수 없으며, 아파트 사이에서 동작하는 특성상 정확도가 떨어지는 현상이 보임.

- (퇴사 당시 진행) VO를 혼합한 위치 표기 알고리즘 개발

- VO를 활용한 알고리즘이 정성적으로 평가하였을 때 더 높은 성능을 보임.
- 해질녘에 촬영하여 밝은 이미지가 어두워 지는 양상을 띄게 될 경우 정확도가 급격히 떨어지는 현상 발견 이를 GPS를 활용해 보정하려는 연구를 진행하던 중 퇴사

성과

- 최초 학습 파이프라인 설계
- 균열위치 표기 알고리즘 개발 및 논문 작성



그림 6. 결과물 예시

